

Retrieval Design for Real-World RAG

Malawi Integrated Disease Surveillance and Response dataset

Assignment Q1 | Local-only pipeline | September 2026

Design decision. Use structure-aware chunks and hybrid BM25 + vector retrieval, then fuse the two ranked lists locally. This protects exact disease terminology and paragraph references while still matching paraphrased questions.

1. Dataset and retrieval objective

The corpus contains six Excel workbooks derived from Malawi IDSR training guides. Column A holds a stable paragraph number and column B holds the corresponding guideline text. The retrieval layer should return the smallest evidence set that fully answers a question while preserving enough provenance for a reviewer to locate the statement in the original booklet.

The current ingestion produces 1,036 chunks from the six workbooks. Chunk lengths are validated during ingestion; the current maximum complete chunk is 4,868 characters. These observations support paragraph-aware splitting rather than a fixed token window over flattened workbook text.

2. Structure-aware chunking

Headings. Detect numbered headings, SECTION/CHAPTER/ANNEX labels, and short uppercase headings. A chunk never crosses a detected section boundary, and the active heading is prepended to its text so the embedding retains topic context.

Lists and small sections. Keep each workbook row as an atomic paragraph whenever possible. Consecutive short rows, bullets, and list items remain together under their heading; this prevents a list marker or condition from being separated from the statement it qualifies.

Long sections. Accumulate rows toward approximately 1,800 characters and cap source content near 2,600 characters. Split an oversized cell at sentence boundaries and carry one source paragraph into the next long-section chunk when needed for local continuity. Preserve the original paragraph number even when one cell becomes multiple chunks.

3. Indexing and metadata

Index the same normalized chunks in two stores: normalized MiniLM embeddings in a persistent Chroma cosine collection and token statistics in a local BM25 index. Keeping one shared chunk identity makes fusion, filtering, tracing, and citation validation deterministic.

Metadata field	Purpose
chunk_id	Stable citation key: booklet + paragraph range + sequence.
doc_id / source_file	Booklet filter and original workbook provenance.
section / topic	Heading context, display label, and topical filtering.
paragraph_start / end	Exact evidence location and trace display.
text	Normalized heading plus source paragraphs used for retrieval.

4. Retrieval and local reranking

Run semantic and lexical retrieval in parallel. Vector search captures paraphrases such as “community reporting” versus “community-based surveillance.” BM25 protects exact matches for disease names, acronyms, case definitions, specimen terms, and paragraph language. Vector-only retrieval is therefore insufficient for this mixed technical corpus; hybrid search improves robustness without requiring an external service.

Candidate fusion. Retrieve at least 20 candidates from each index. Apply any doc_id, section, or topic filters before ranking. Fuse the lists using reciprocal-rank fusion: $RRF(d) = \text{sum over retrievers of } 1 / (60 + \text{rank}(d))$. RRF is the local reranker: it rewards chunks supported by both retrieval signals without pretending cosine and BM25 scores share a calibrated scale.

Choosing top_k. Use k = 6 chunks for answer generation as the current operating point, while each retriever produces at least 20 candidates before fusion. Calibrate the value on the golden set against recall, context size, latency, and faithfulness; increase k for comparisons and reduce it for narrow definitions. Select the final value on a 15–20 question golden set by plotting Recall@k against context size, latency, and faithfulness, choosing the smallest k that reaches the recall target.

Measured validation sample. In the three-question local Ragas sample (Q220, Q1016, and Q1226), context precision and context recall were 1.0000 for every question. This supports the current hybrid retrieval configuration for those examples, but it is not a substitute for deterministic Recall@k/MRR on the frozen golden set or evidence that hybrid retrieval always outperforms vector-only search.

Optional second stage. If evaluation shows that fusion alone leaves ambiguous near-duplicates, rerank only the top 12–20 candidates with a small local cross-encoder and pass the best four onward. This is a second-stage option, not the default: reciprocal-rank fusion is faster, fully local, transparent in trace mode, and already implemented.

5. Correctness controls

Gate generation on both semantic similarity and lexical query coverage. The current starting thresholds are cosine similarity ≥ 0.28 and query-token coverage ≥ 0.12 ; they must be calibrated on relevant and deliberately unanswerable questions. When the gate fails, return “cannot find in sources” without calling the LLM.

Send only the selected chunks to the local answer model and require exact bracketed chunk IDs for every factual claim. Reject missing or unknown citations deterministically. Trace mode should expose rank, semantic similarity, BM25 score, fused score, section, paragraph range, and retrieved text so incorrect answers can be traced back to retrieval rather than hidden in generation.

6. Retrieval workflow design

Q1 Retrieval Workflow

OFFLINE INDEX BUILD



ONLINE QUERY AND EVIDENCE SELECTION



Workflow. (1) read workbook rows and stable paragraph numbers; (2) detect section boundaries and normalize text; (3) create paragraph-aware chunks with provenance; (4) write the same chunk IDs to MiniLM/Chroma and BM25 indexes; (5) validate the question and optional metadata filters; (6) run semantic and lexical retrieval; (7) fuse candidates with RRF and apply the evidence gate; (8) return the smallest top-k evidence set to generation, or abstain when support is insufficient. Trace metadata follows every chunk so retrieval errors remain auditable.

The accompanying snippets `q1.py` provides the required `chunk_documents(docs)`, `build_index(chunks)`, and `retrieve(query, filters, k)` interfaces. They call the same modules used by the working Q2 demo, keeping the design and implementation consistent.